Programación Móvil para Profesores de Educación Media Superior

Dr. Marco Aurelio Nuño Maganda

Universidad Politécnica de Victoria
Laboratorio de Sistemas Inteligentes
mnunom@upv.edu.mx

Marzo de 2026

Contenido

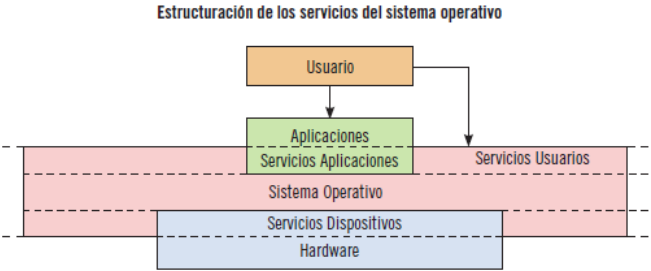
1. Programación Móvil
2. Programación Orientada a Objetos
3. Configuracion de Dispositivo
4. Conceptos Basicos de Apps en Android
5. Demos
6. Conclusiones

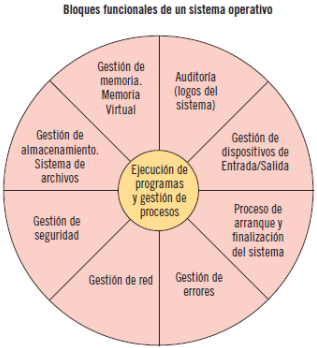
Outline

1. Programación Móvil
2. Programación Orientada a Objetos
3. Configuracion de Dispositivo
4. Conceptos Basicos de Apps en Android
5. Demos
6. Conclusiones

Sistema Operativo

Un Sistema Operativo (SO) es un programa (software) que al arrancar la computadora** se encarga de gestionar todos los recursos del sistema informático permitiendo así la comunicación entre el usuario y la computadora.





https://reader.digitalbooks.pro/content/preview/books/38230/book/OEBPS/Text/c1.html

Sistemas Operativos para PCs

Telefono Celular No-inteligente vs Telefono Celular Inteligente

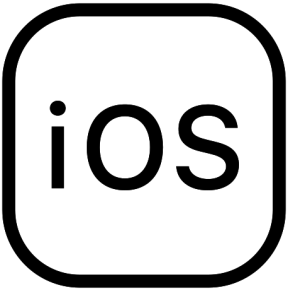
Teléfono No-inteligente

- Su funcionalidad principal era la comunicación (llamadas o mensajes) a través de la red celular (GSM)

Teléfono inteligente

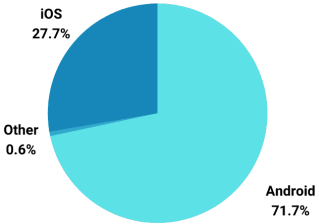
- Interfaz de entrada: Pantalla Touch (a color, de alta definición)
- Conexión a Internet: WiFi, GSM (4G o 5G)
- Comunicación con otros dispositivos: Bluetooth, NFC
- Cámaras (Frontal y Posterior)

Sistemas Operativos para Telefonos Inteligentes



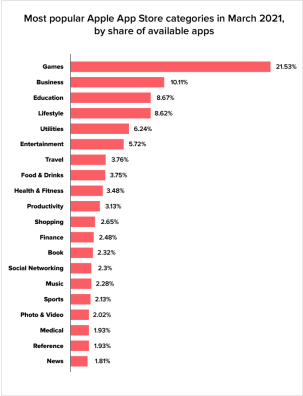
Android

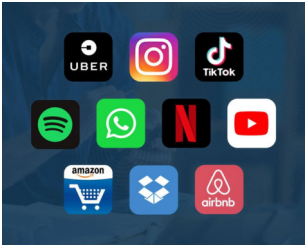
- Android es un sistema operativo móvil basado en Linux
- Principalmente orientado a dispositivos de pantalla táctil (Smartphone, tablets, smartwatches, etc)
- Fue desarrollado por Android Inc (Adquirida por Google en 2005)
- Vinculado con un grupo de empresas (HTC, Sony, Motorola, Samsung, LG, Lenovo, entre otras) para la creación de un SO común para sus dispositivos
- A la fecha (Q1 2023), los teléfonos con SO Android concentran mas del 70% del mercado global.



Aplicaciones Móviles

- Ejecutadas en el teléfono
- La entrada de datos es mediante un teclado "virtual"
- El apuntador del raton es la pantalla
- Incluyen una interfaz de usuario gráfica (GUI)
- Es posible descargar miles de éstas en nuestros dispositivos





https://www.netsolutions.com/insights/top-10-most-popular-apps-2018/

Outline

1. Programación Móvil
2. Programación Orientada a Objetos
3. Configuracion de Dispositivo
4. Conceptos Basicos de Apps en Android
5. Demos
6. Conclusiones

¿Qué es la Programación Orientada a Objetos?

- Paradigma de programación basado en el uso de **objetos**.
- Los objetos representan entidades con:
  - **Atributos** (datos)
  - **Métodos** (comportamientos)
- Facilita la reutilización, mantenimiento y escalabilidad del código.

Principios Fundamentales de la POO

1. Encapsulamiento

Agrupa datos y métodos en una sola unidad (clase) y restringe el acceso directo.

2. Abstracción

Permite representar conceptos complejos mostrando solo lo esencial.

3. Herencia

Permite que una clase herede propiedades y comportamientos de otra.

4. Polimorfismo

Permite que diferentes objetos respondan de manera distinta al mismo método.

Ejemplo Conceptual

- Clase: `Vehículo`
- Atributos: color, velocidad
- Método: acelerar()
- Subclase: `Auto`, `Motocicleta`

Idea clave: Modelar problemas del mundo real mediante objetos.

Outline

1. Programación Móvil
2. Programación Orientada a Objetos
3. Configuracion de Dispositivo
4. Conceptos Basicos de Apps en Android
5. Demos
6. Conclusiones

Materiales Necesarios

Hardware:

- Cualquier smartphone o tablet con Sistema Operativo Android
- Cable USB de Datos.
- Laptop con cualquier Sistema Operativo (minimo de memoria requerido: 8 GB).

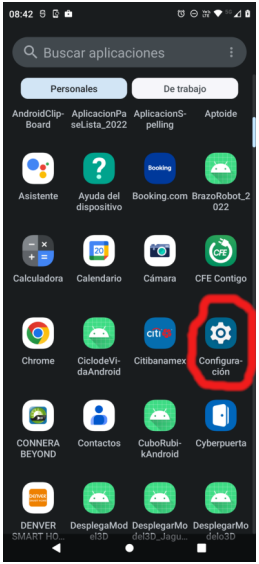
Software:

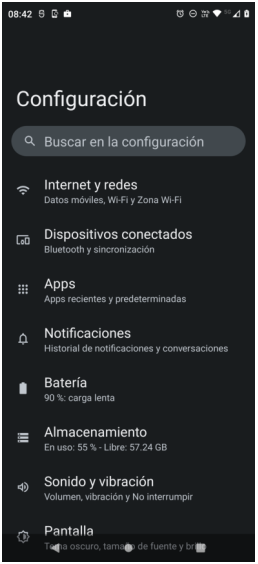
- Android Studio (la versión más reciente).

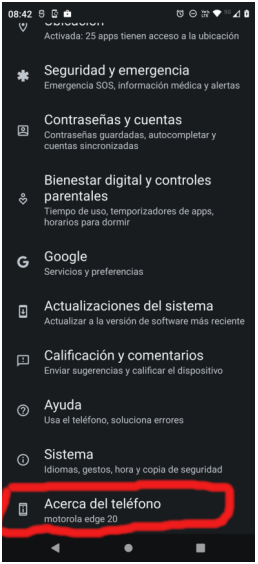
Configurar telefono inteligente en modo de desarrollador (0)

- Es necesario buscar en las opciones de configuración, en el apartado acerca del telefono ubicar el número de compilación
- Dar 5 taps (toques) sobre el número compilación. Debera aparecer un mensaje que diga que estas a X taps de ser un desarrollador
- Lo anterior habilita un nuevo menú en el apartado sistema dentro de opciones de configuración con título "opciones para desarrolladores"
- Debe habilitarse tanto "opciones para desarrolladores" como la opción "depuración USB"

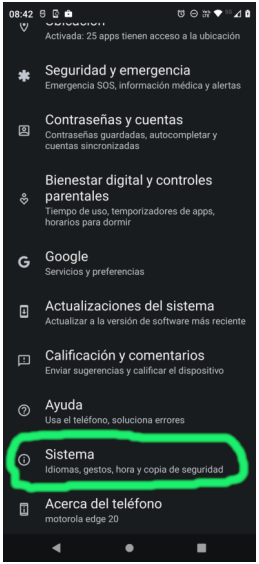
Configurar telefono inteligente en modo de desarrollador (1)

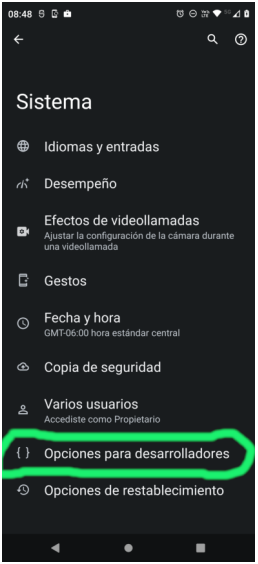


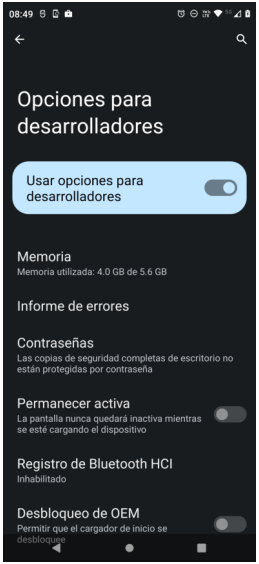


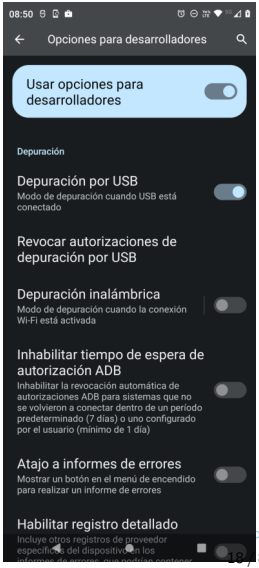


Configurar telefono inteligente en modo de desarrollador (2)







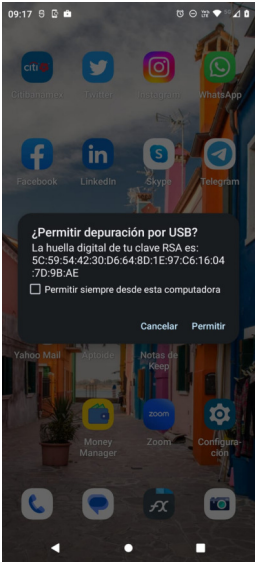


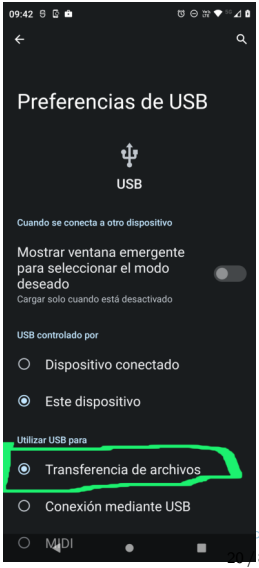
Configurar telefono inteligente en modo de desarrollador (3)

- Al habilitar la depuracion, aparece un mensaje de advertencia
- Una vez que hayas terminado tus pruebas, se recomienda deshabilitar este modo de depuración
- Para poder instalar una aplicación desarrollada con Android Studio, se debe conectar el teléfono inteligente a la computadora usando un cable USB

Conectar smartphone a la computadora (Cable USB)

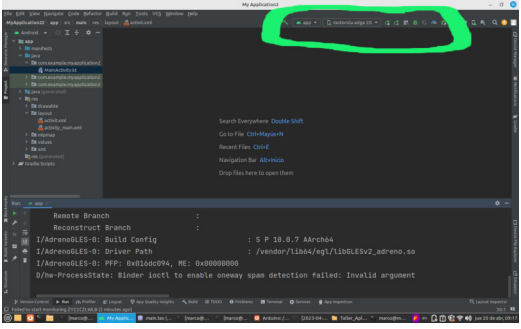
- Al conectar el dispositivo por primera vez, aparece un mensaje de confirmación en el dispositivo
- Configurar el telefono para que se conecte en modo de transferencia de archivos (por defecto, esta cargando)

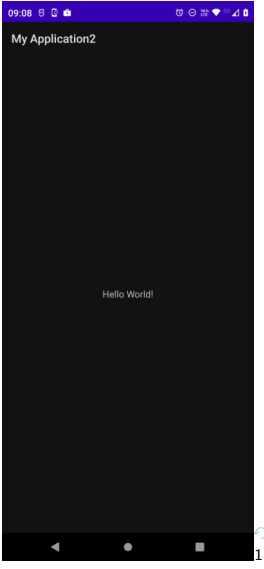




Ejecutar proyecto en telefono inteligente

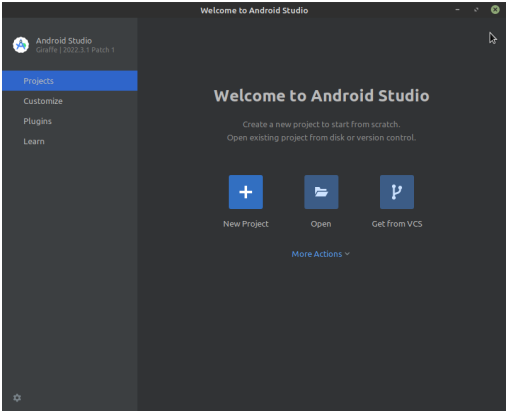
- Una vez conectado el telefono, debe aparecer el modelo en la parte superior (lado derecho) de tu proyecto de Android Studio
- Para instalar la aplicacion en el telefono, deber dar click en una flecha verde justo a un lado de nombre del telefono (sean pacientes, puede tardar un poco la primera vez)





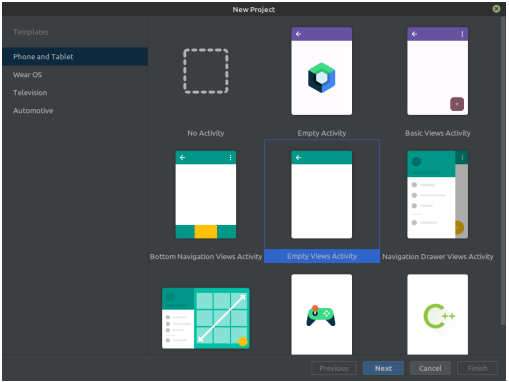
Crear un primer proyecto (1)

- Buscar el icono de la aplicacion y dar doble click
- En caso de que requiera algunas actualizaciones, ser paciente, puede tardar.
- Aparecera una ventana como la mostrada, seleccionar la opción “New project”



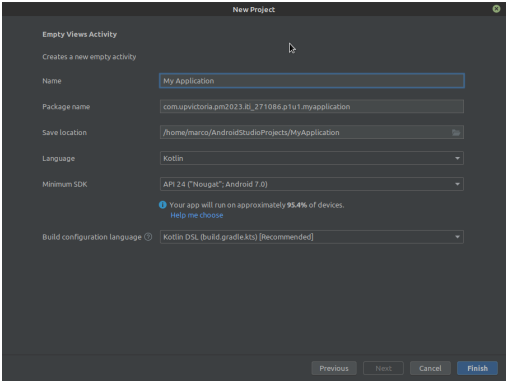
Crear un primer proyecto (2)

- Existen varios tipos de dispositivos para los que pueden crearse proyectos. Por defecto nos ubica en el primer grupo (Phone and Tablet)
- Seleccionar tipo de proyecto "Empty Views Activity" y dack click en Next



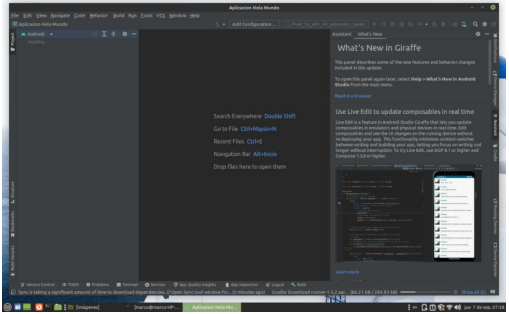
Crear un primer proyecto (3)

- Dejar los valores por defecto, solo se suguiere cambiar el nombre del proyecto
- **Cambiar la seleccion de lenguaje a Java** - por defecto esta seleccionado Kotlin.
- Presionar el boton Finish



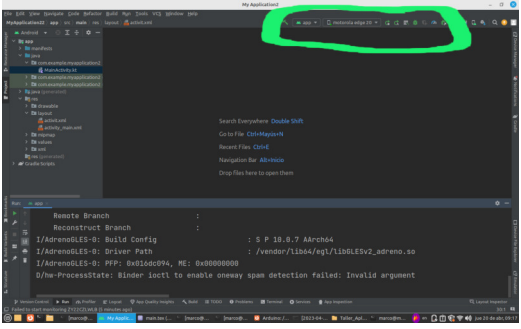
Crear un primer proyecto (4)

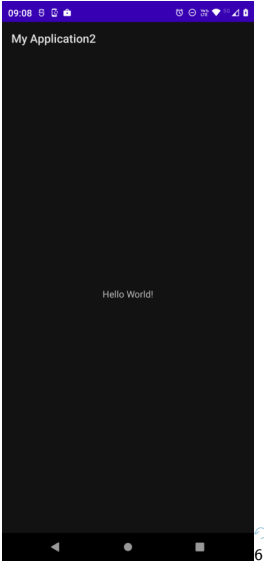
- La primera vez que se crea un proyecto, puede requerir la descarga de archivos necesarios. Le recomiendo ser paciente
- Ya una vez que el proyecto fue creado, aparece la ventana mostrada



Ejecutar proyecto en telefono inteligente

- Una vez conectado el telefono, debe aparecer el modelo en la parte superior (lado derecho) de tu proyecto de Android Studio
- Para instalar la aplicacion en el telefono, deber dar click en una flecha verde justo a un lado de nombre del telefono (sean pacientes, puede tardar un poco la primera vez)





Outline

1. Programación Móvil
2. Programación Orientada a Objetos
3. Configuracion de Dispositivo
4. Conceptos Basicos de Apps en Android
5. Demos
6. Conclusiones

Componentes de una APP en Android

¿Qué es un Layout?

- Es un contenedor que define la estructura visual de la interfaz de usuario.
- Organiza y posiciona los componentes (Views) como botones, textos e imágenes.
- Se puede definir en archivos XML (por defecto, AS genera un archivo *activity_main.xml*).

Tipos principales de Layouts:

- **LinearLayout:** Organiza los elementos en una fila o columna.
- **GridLayout:** Organiza los elementos en una cuadrícula.
- **RelativeLayout:** Posiciona elementos en relación a otros componentes.
- **ConstraintLayout:** Permite diseños flexibles usando restricciones.
- **FrameLayout:** Superpone elementos uno encima de otro.

Componentes de Interfaz de usuario de una APP en Android

¿Qué es TextView?

- Es un componente de interfaz de usuario utilizado para mostrar texto en una aplicación Android.
- Permite presentar información estática o dinámica al usuario.
- Soporta texto plano y con formato (colores, tamaños, estilos).

Características principales:

- Soporta texto plano y con formato (colores, tamaños, estilos).
- Puede configurarse desde XML o mediante código Java/Kotlin.
- Permite alineación, padding y control de tamaño de texto.

Actividades e Intents en Android

¿Qué es una Actividad (Activity)?

- Es un componente fundamental de una aplicación Android que representa una pantalla con la que el usuario puede interactuar.

Principales tipos de Actividades:

- **Actividad Principal (MainActivity):** Punto de entrada de la aplicación.
- **Actividades Secundarias:** Pantallas adicionales para navegación y funciones específicas.
- **Actividades de Diálogo:** Se muestran como ventanas emergentes.

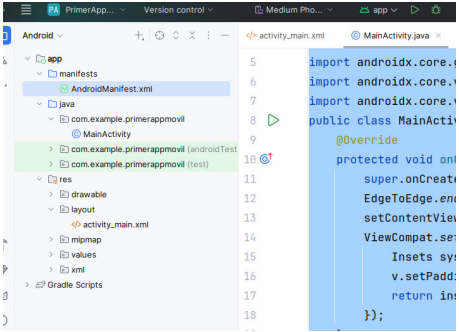
¿Qué es un Intent (Intento)?

- Es un mecanismo de comunicación entre componentes de Android, que permite lanzar actividades (de sistema o codificadas por el usuario.
- Puede ser **explícito** (define el componente destino) o **implícito** (define una acción a realizar).

Carpetas del proyecto

Carpetas y archivos importantes

- *Java* - Código fuente - Dentro hay un archivo *MainActivity.java*
- *Res* - Recursos (imágenes) y definición de interfaces. Dentro hay una carpeta llamada *layout*, con un archivo *activity_main.xml*
- *Manifest* - Contiene actividades y permisos que la app require. Tambien define cual es es la app de entrada



Agregar codigo a la interfaz

- X

Apariencia

Archivo **activity_main.xml**

```xml
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Agregar codigo a la interfaz

Comportamiento

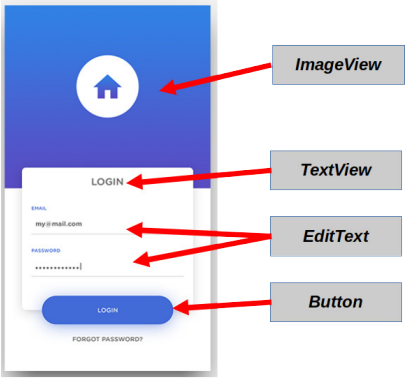
- X

Archivo **MainActivity.kt**

```java
package com.example.primerappmovil;
import android.os.Bundle;
import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
    }
```

Interfaz de Usuario

- Define los controles de interfaz que la aplicacion muestra
- Estos controles pueden estar agrupados en contenedores
- El archivo **activity_main.xml** es la definicion del layout (la organizacion de dichos componentes)



Outline

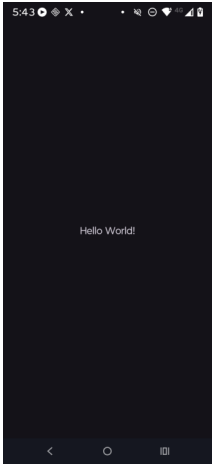
1. Programación Móvil
2. Programación Orientada a Objetos
3. Configuracion de Dispositivo
4. Conceptos Basicos de Apps en Android
5. Demos
6. Conclusiones

Ejecucion de la Primer App en Android

- El dispositivo debe estar reconocido por Android y la compilación de la primera APP debe haber terminado.
- En la barra superior de Android Studio, presionar la flecha verde para instalar y ejecutar la app en el disposivito,

Demo #00

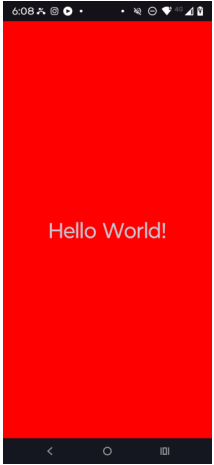
Muestra un TextView con el texto centrado "Hello World!"



Demo 01: Modificar el activity_main.xml para aregar otro TextView

De los códigos fuentes en Github (como un archivo .zip), descomprimirlo y localizar los archivos mencionados a continuación:

- Copiar el archivo *MainActivity_Demo01.java* (dentro del Folder códigosJAVA) al mismo directorio donde esta el *MainActivity.java*.
- Copiar el archivo *activity_main_demo01.xml* (dentro del Folder Carpeta_layout) al mismo directorio donde esta el *activity_main.xml*
- Modificar en el archivo AndroidManifest.xml la cadena *.MainActivity* por *.MainActivity_Demo01*.



Demo #01

Muestra un TextView con el texto centrado "Hello World!" (fuente verde, fondo rojo)

Modificaciones de Interés

Ejercicio #01

- Duplicar el TextView pero modificando el color de dichos componentes.
- Probar diferentes combinaciones del valor *layout_weight* en dichos componentes.

Componentes EditText y Button en Android

EditText: Campo de Entrada de Texto

- Permite al usuario introducir y editar texto dentro de la aplicación.
- Se utiliza en formularios, búsquedas y captura de datos.
- Soporta distintos tipos de entrada (texto, número, correo, contraseña).
- Puede configurarse mediante atributos XML o código Java/Kotlin.

Button: Botón de Interacción

- Es un componente que permite al usuario ejecutar acciones al hacer clic.
- Se utiliza para enviar datos, navegar entre pantallas o activar eventos.
- Puede personalizarse en texto, color, tamaño y estilo.

Demo 02: Agregar un EditText y un Button

De los códigos fuentes en Github (como un archivo .zip), descomprimirlo y localizar los archivos mencionados a continuación:

- Copiar el archivo *MainActivity_Demo02.java* (dentro del Folder códigosJAVA) al mismo directorio donde esta el *MainActivity.java*.
- Copiar el archivo *activity_main_demo02.xml* (dentro del Folder Carpeta_layout) al mismo directorio donde esta el *activity_main.xml*.
- Modificar en el archivo *AndroidManifest.xml* la cadena *..MainActivity_Demo01* por *.MainActivity_Demo02*

Demo #02

Muestra una App con dos TextViews, un EditText y un Botton. Cuando el usuario ingresa un dato y presiona le boton aparece un mensaje con el dato ingresado

Uso de GridLayout en Android

¿Qué es GridLayout?

- Es un contenedor de diseño (ViewGroup) que organiza los elementos en una cuadrícula de filas y columnas.
- Permite ubicar componentes de manera estructurada y alineada.

¿Para qué sirve?

- Diseñar interfaces organizadas en forma de tabla.
- Alinear múltiples elementos (botones, imágenes, textos) fácilmente.
- Crear interfaces dinámicas como teclados, galerías o tableros de juego.

Características principales:

- Define número de filas y columnas.
- Permite que los elementos ocupen múltiples celdas (rowSpan, columnSpan).
- Soporta alineación y distribución del espacio.

Demo 03: Agregar un Panel de Botones utilizando un GridLayout

Del Repositorio Github:

- Copiar el archivo MainActivity_Demo03.java (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo activity_main_demo03.xml (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml
- Modificar en el archivo AndroidManifest.xml la cadena *.MainActivity_Demo02* por *.MainActivity_Demo03*

Demo #03

Muestra una App con un panel de botones, que muestra un Toast para cada boton presionado por el usuario.

Componentes CheckBox e ImageView en Android

CheckBox: ¿Para qué sirve?

- Permite al usuario seleccionar una o múltiples opciones de una lista.
- Cada opción es independiente (no excluyente).
- Es útil en formularios, configuraciones o encuestas.

Características del CheckBox:

- Puede estar en estado seleccionado o no seleccionado.
- Permite manejar eventos de cambio (OnCheckedChangeListener).

ImageView: ¿Para qué sirve?

- Se utiliza para mostrar imágenes en la interfaz de usuario.
- Soporta recursos locales (drawable) o imágenes desde URL.

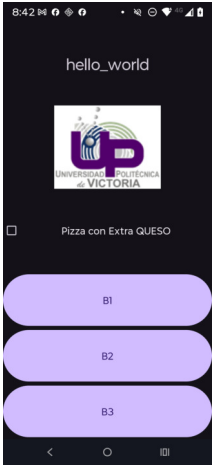
Características del ImageView:

- Permite escalar, ajustar y posicionar imágenes.
- Compatible con diferentes formatos (PNG, JPG, SVG - mediante librerías).

Demo 04: Agregar un CheckBox y un ImageView

Del Repositorio Github:

- Copiar el archivo MainActivity_Demo04.java (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo activity_main_demo04.xml (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml
- Copiar el archivo *upv.png* dentro de la folder *Carpeta_drawable* en el directorio *res/drawable* del proyecto de AS.
- Modificar en el archivo AndroidManifest.xml la cadena *.MainActivity_Demo03* por *.MainActivity_Demo04*



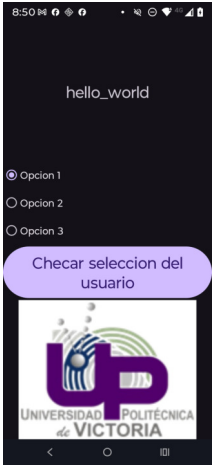
Demo #04

Una App con in Checkbox y un ImageView. Al modificar el estado del CheckBox, el texto de un Textview cambia. Al presionar el primer boton, un Toast muestra el estado del Checkbox.

Demo 05: Agregar un RadioGroup

Del Repositorio Github:

- Copiar el archivo MainActivity_Demo05.java (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo activity_main_demo05.xml (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml
- Copiar los archivos *cgup.jpg* y *logo_upv_2019.png* localizados dentro de la folder *Carpeta_drawable* en el directorio *res/drawable* del proyecto de AS.
- Modificar en el archivo AndroidManifest.xml la cadena *.MainActivity_Demo04* por *.MainActivity_Demo05*



Demo #05

Muestra una App con un RadioGroup. El ImageView cambia basado en la seleccion del usuario.

Componentes Adapter y ListView en Android

¿Qué es ListView?

- Es un componente de interfaz que permite mostrar una lista desplazable de elementos.
- Cada elemento puede contener texto, imágenes u otros componentes.
- Es útil para mostrar datos en forma de lista vertical.

¿Qué es un Adapter?

- Es un intermediario entre los datos y el ListView.
- Se encarga de convertir los datos (arreglos, listas, bases de datos) en vistas visuales.
- Permite personalizar cómo se muestra cada elemento de la lista.

Relación entre Adapter y ListView:

- El ListView solicita los datos al Adapter.
- El Adapter genera la vista de cada elemento.

Componentes Adapter y ListView en Android

¿Qué es ListView?

- Es un componente de interfaz que permite mostrar una lista desplazable de elementos.
- Cada elemento puede contener texto, imágenes u otros componentes.
- Es útil para mostrar datos en forma de lista vertical.

¿Qué es un Adapter?

- Es un intermediario entre los datos y el ListView.
- Se encarga de convertir los datos (arreglos, listas, bases de datos) en vistas visuales.
- Permite personalizar cómo se muestra cada elemento de la lista.

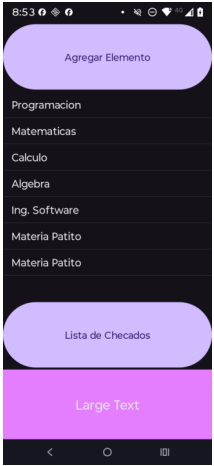
Relación entre Adapter y ListView:

- El ListView solicita los datos al Adapter.
- El Adapter genera la vista de cada elemento.

Demo 06: Agregar un ListView

Del Repositorio Github:

- Copiar el archivo MainActivity_Demo06.java (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo activity_main_demo06.xml (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml
- Modificar en el archivo AndroidManifest.xml la cadena *.MainActivity_Demo05* por *.MainActivity_Demo06*



Demo #06

Muestra una App con un ListView. El usuario agregar elementos al presionar el boton superior.

Spinner vs ListView en Android

¿Qué es un Spinner?

- Es un componente que permite seleccionar una opción de una lista desplegable.
- Ocupa poco espacio en pantalla.
- Ideal cuando se requiere elegir una sola opción entre varias.

¿Qué es un ListView?

- Es un componente que muestra una lista de elementos visibles en pantalla.
- Permite visualizar múltiples opciones al mismo tiempo.
- Puede soportar selección simple o múltiple.

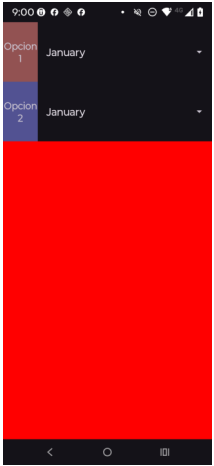
Diferencias principales:

- **Espacio:** Spinner es compacto, ListView requiere más espacio.
- **Interacción:** Spinner usa menú desplegable, ListView muestra lista directa.
- **Uso:** Spinner para selección rápida; ListView para exploración de datos.
- **Cantidad de datos:** ListView es más adecuado para listas largas.

Demo 07: Agregar un Spinner

Del Repositorio Github:

- Copiar el archivo MainActivity_Demo07.java (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo activity_main_demo07.xml (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml
- Modificar en el archivo AndroidManifest.xml la cadena *.MainActivity_Demo06* por *.MainActivity_Demo07*



Demo #07

Muestra una App con un ListView. El usuario agregar elementos al presionar el boton superior.

Componente SeekBar en Android

¿Qué es un SeekBar?

- Es un componente de interfaz gráfica que permite al usuario seleccionar un valor dentro de un rango deslizando un control horizontal.
- Es una extensión de la clase ProgressBar con interacción del usuario.

¿Para qué sirve?

- Ajustar valores continuos o discretos (ej. volumen, brillo, progreso).
- Permitir entrada intuitiva mediante deslizamiento.
- Controlar parámetros en tiempo real.

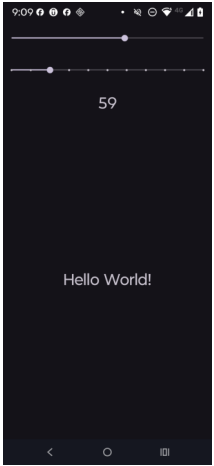
Características principales:

- Permite definir valores mínimo, máximo y progreso actual.
- Puede escuchar cambios mediante un listener (OnSeekBarChangeListener).
- Es altamente personalizable (colores, tamaño, estilo).

Demo 08: Agregar SeekBars

Del Repositorio Github:

- Copiar el archivo MainActivity_Demo08.java (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo activity_main_demo08.xml (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml
- Modificar en el archivo AndroidManifest.xml la cadena *.MainActivity_Demo07* por *.MainActivity_Demo08*



Demo #08

Muestra una App dos SeekBars (la segunda utiliza separadores finitos). La primera SeekBar actualiza el contenido de TextView conforme la seleccion del usuario.

startActivityForResult y onActivityResult

¿Para qué sirven?

- Permiten iniciar una actividad esperando un resultado de regreso.
- Son útiles cuando se necesita que una segunda pantalla devuelva información (por ejemplo, selección de datos o confirmaciones).

Flujo de funcionamiento:

1. Se inicia la actividad con `startActivityForResult()`.
2. La segunda actividad procesa la información.
3. Se devuelve un resultado mediante `setResult()`.
4. La actividad original recibe el resultado en `onActivityResult()`.

Ejemplo en Java:

```java
// Lanzar actividad
Intent intent = new Intent(this, SegundaActivity.class);
startActivityForResult(intent, 1);

// Recibir resultado
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    if (requestCode == 1 && resultCode == RESULT_OK) {
        String resultado = data.getStringExtra("dato");
    }
}
```

Lectura de Archivos en Android (Java)

Método: readFromFile(Uri uri, String text)

- Este método permite leer el contenido de un archivo a partir de una **URI**.
- Utiliza el **ContentResolver** para acceder a recursos externos.

Flujo del código:

1. Se obtiene un flujo de entrada:
   - `InputStream is = getContentResolver().openInputStream(uri);`
2. Se crea un lector de texto:
   - `BufferedReader bf = new BufferedReader(new InputStreamReader(is));`
3. Se lee el archivo línea por línea:
   - Se usa un ciclo `while` para concatenar el contenido en una variable.
4. Se muestra el contenido en pantalla:
   - Se obtiene un `TextView` con `findViewById`.
   - Se asigna el texto con `setText(FileContents)`.

Manejo de errores:

- Se utiliza un bloque `try-catch` para capturar excepciones `IOException`.

Escritura de Archivos en Android

Método: writeInFile(...)

- Este método permite escribir texto en un archivo identificado por una URI.
- Utiliza flujos de salida (*OutputStream*) para manejar la escritura.

Parámetros:

- **Uri uri:** Dirección del archivo donde se escribirá el contenido.
- **String text:** Texto que se desea guardar.

Funcionamiento:

- Se obtiene un **OutputStream** mediante `getContentResolver()`.
- Se envuelve en un **BufferedWriter** para escribir texto eficientemente.
- Se escribe el contenido con `write(text)`.
- Se asegura la escritura con `flush()`.
- Se cierra el flujo con `close()`.

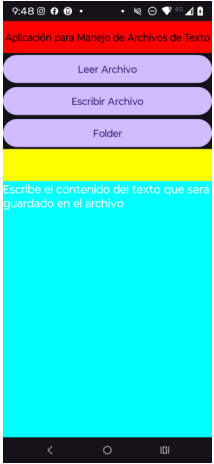
Manejo de errores:

- Se utiliza un bloque `try-catch` para capturar excepciones de tipo `IOException`.

Demo 09: Lectura y Escritura de Archivos de Texto

Del Repositorio Github:

- Copiar el archivo *MainActivity_Demo09.java* (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo *activity_main_demo09.xml* (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml
- Modificar en el archivo *AndroidManifest.xml* la cadena *.MainActivity_Demo08* por *.MainActivity_Demo09*.
- Ir a la linea 22 de archivo del archivo *MainActivity_Demo09.java*, poner el cursos en la palabra (en Rojo) *documentfile*, presionar la combinación ALT+ENTER, y seleccionar de la lista desplegable la opcion "Add dependency on ....". Esto quitará el error y hara los ajustes necesarios.



ProgressDialog en Android

¿Qué es ProgressDialog?

- Es un componente utilizado para mostrar un indicador de progreso al usuario.
- Se emplea comúnmente durante operaciones largas (carga de datos, procesos en segundo plano).

Características:

- Puede mostrar un mensaje informativo.
- Puede ser cancelable o no por el usuario.
- Soporta estilos como barra de progreso o spinner.

Uso de AsyncTask para tareas en segundo plano

Clase: DownloadTask extends AsyncTask<String, Integer, List<String>>

- Permite ejecutar tareas en segundo plano sin bloquear la interfaz gráfica.
- Usa tres tipos genéricos:
  - **String:** parámetros de entrada.
  - **Integer:** progreso.
  - **List<String>:** resultado final.

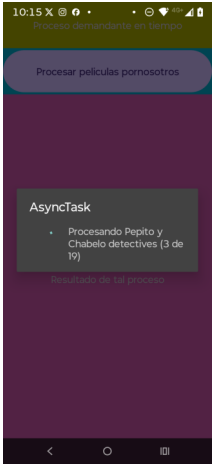
Fases del AsyncTask:

- **onPreExecute():** Inicializa variables y muestra el ProgressDialog.
- **doInBackground():** Ejecuta tareas en segundo plano, publica progreso y simula espera.
- **onProgressUpdate():** Actualiza la barra de progreso y mensaje.
- **onPostExecute():** Oculta el diálogo y muestra resultados en pantalla.

Demo 10: Progress Dialog y AsyncTask

Del Repositorio Github:

- Copiar el archivo *MainActivity_Demo10.java* (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo *activity_main_demo10.xml* (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml
- Modificar en el archivo *AndroidManifest.xml* la cadena *.MainActivity_Demo09* por *.MainActivity_Demo10*.



AlertDialog.Builder y Dialog con XML en Android

¿Qué es un Cuadro de Dialogo?

- Es una ventana emergente que permite interactuar con el usuario.

¿Qué es AlertDialog.Builder?

- Clase utilizada para construir y mostrar diálogos emergentes.
- Permite definir título, mensaje, botones y eventos.

Dialog con contenido XML:

- Permite usar una interfaz personalizada definida en un layout XML.
- Se establece su contenido en base al archivo XML.

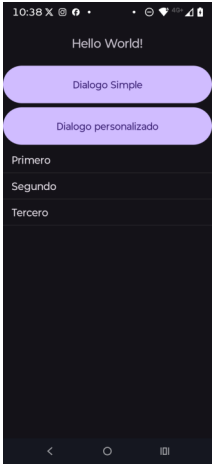
Ventajas del Dialogo Personalizado:

- Mayor personalización de la interfaz.
- Permite incluir EditText, imágenes, botones, etc.

Demo 11: Dialog y CustoDialog

Del Repositorio Github:

- Copiar el archivo *MainActivity_Demo11.java* (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo *activity_main_demo11.xml* (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml
- Copiar el archivo *custom_dialog.xml* (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml.
- Modificar en el archivo *AndroidManifest.xml* la cadena *.MainActivity_Demo10* por *.MainActivity_Demo11*.



Uso de TabHost y TabWidget en Android

¿Qué son TabHost y TabWidget?

- **TabHost:** Contenedor principal que gestiona múltiples pestañas (tabs).
- **TabWidget:** Componente visual que muestra las pestañas en la interfaz.
- Permiten organizar contenido en diferentes secciones dentro de una misma Activity.

Estructura básica:

- TabHost contiene:
  - Un **TabWidget** (las pestañas).
  - Un **FrameLayout** (contenido de cada tab).

Demo 12: Pestañas (Tabs)

Del Repositorio Github:

- Copiar el archivo *MainActivity_Demo12.java* (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo *activity_main_demo12.xml* (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml
- Modificar en el archivo *AndroidManifest.xml* la cadena *.MainActivity_Demo11* por *.MainActivity_Demo12*.

Lectura de Códigos QR en Android

¿Qué es un decodificador QR?

- Permite leer y decodificar información contenida en códigos QR usando la cámara del dispositivo.
- Se utiliza en aplicaciones de pagos, accesos, inventarios, entre otros.

Uso de librerías externas:

- Android no incluye un lector QR nativo completo.
- Se pueden integrar librerías como **ZXing** o **ML Kit**.

```
dependencies {
    implementation 'com.journeyapps:zxing-android-embedded:4.3.0'
    implementation 'com.google.zxing:core:3.5.1'
}
```

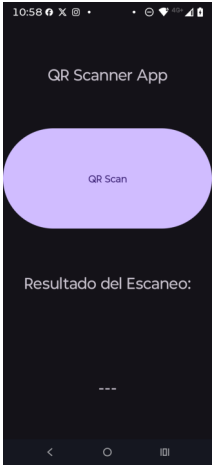
Ventajas:

- Integración rápida y sencilla.
- Soporte para múltiples formatos de códigos.

Demo 13: Decodificacion de Codigos QR con Camara

Del Repositorio Github:

- Copiar el archivo *MainActivity_Demo13.java* (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo *activity_main_demo13.xml* (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml
- Modificar en el archivo *AndroidManifest.xml* la cadena *.MainActivity_Demo12* por *.MainActivity_Demo13*.
- Agregar la siguiente linea: *implementation ("com.journeyapps:zxing-android-embedded:4.3.0")*, al final de la seccion Dependencies del archivo Build.graddle.kts (Module: App).



Uso de Text-to-Speech (TTS) en Android

¿Qué es Text-to-Speech (TTS)?

- Es una tecnología que convierte texto en voz sintetizada.
- Permite a una aplicación "hablar" al usuario.
- Útil en accesibilidad, asistentes virtuales y notificaciones.

Pasos básicos:

- Crear una instancia de `TextToSpeech`.
- Configurar idioma y velocidad.
- Invocar el método `speak()` para reproducir el texto.

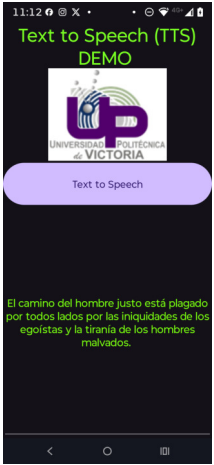
Consideraciones:

- Verificar que el idioma esté soportado.
- Liberar recursos con `tts.shutdown()` en `onDestroy()`.
- Requiere motor TTS instalado en el dispositivo.

Demo 14: Text to Speech (TTS)

Del Repositorio Github:

- Copiar el archivo *MainActivity_Demo14.java* (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo *activity_main_demo14.xml* (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml
- Modificar en el archivo *AndroidManifest.xml* la cadena *.MainActivity_Demo13* por *.MainActivity_Demo14*.



Librería MPAndroidChart en Android

¿Qué es MPAndroidChart?

- Es una librería para la visualización de datos mediante gráficas en Android.
- Permite crear gráficos interactivos y altamente personalizables.

Tipos de gráficas soportadas:

- Gráficas de línea (LineChart)
- Barras (BarChart)
- Pastel (PieChart)
- Dispersión (ScatterChart)

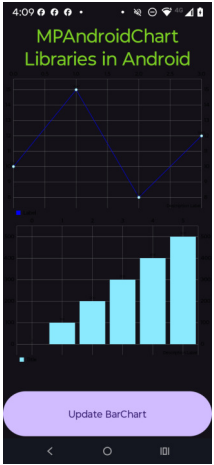
Ventajas:

- Fácil integración.
- Alta personalización (colores, animaciones).
- Soporte para interacción (zoom, desplazamiento).

Demo 15: Graficas con MPAndroid

Del Repositorio Github:

- Agregar la siguiente linea: *implementation("com.github.PhilJay:MPAndroidChart:v3.1.0")*, al final de la seccion Dependencies del archivo Build.graddle.kts (Module: App).
- Agregar la siguiente linea: *maven { url = uri("https://jitpack.io") }*, al final de la seccion *dependencyResolutionManagement* del archivo settings.graddle.kts.
- Copiar el archivo *MainActivity_Demo15.java* (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo *activity_main_demo15.xml* (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml
- Modificar en el archivo *AndroidManifest.xml* la cadena *.MainActivity_Demo14* por *.MainActivity_Demo15*.



Uso de Canvas View en Android

¿Qué es un Canvas View?

- Permite dibujar gráficos personalizados directamente en pantalla.
- Se utiliza extendiendo la clase `View` y sobrescribiendo el método `onDraw()`.
- Ideal para juegos, visualizaciones y dibujo libre.

Métodos principales:

- **onDraw(Canvas canvas):** Método donde se realiza el dibujo.
- **invalidate():** Solicita redibujar la vista.
- **onTouchEvent():** Maneja interacción táctil.

Clases importantes:

- **Canvas:** Superficie de dibujo.
- **Paint:** Define color, estilo y grosor.
- **Path:** Permite dibujar formas complejas.

Ventajas:

- Alto control sobre gráficos.
- Ideal para interfaces personalizadas.

Demo 16: CanvasView

Del Repositorio Github:

- Copiar el archivo *MainActivity_Demo16.java* (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo *CanvasView.java* (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo *activity_main_demo16.xml* (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml
- Modificar dentro del archivo *activity_main_demo16.xml* la cadena *.com.example.a2026_01_holamundoandroid.CanvasView* por *<SuPackage>.CanvasView*.
- Modificar en el archivo *AndroidManifest.xml* la cadena *.MainActivity_Demo15* por *.MainActivity_Demo16*.

Uso de SurfaceView en Android

¿Qué es SurfaceView?

- Es un componente que permite dibujar directamente sobre una superficie separada del hilo principal (UI thread).
- Ideal para gráficos en tiempo real como videojuegos, animaciones o streaming de video.

Componentes clave:

- **SurfaceHolder:** Controla la superficie de dibujo.
- **Canvas:** Permite realizar los dibujos.
- **Thread:** Maneja el ciclo de renderizado.

Métodos principales (SurfaceHolder.Callback):

- **surfaceCreated():** Se invoca cuando la superficie está lista para dibujar.
- **surfaceChanged():** Se ejecuta cuando cambia tamaño o formato.
- **surfaceDestroyed():** Se llama cuando la superficie se destruye.

Ventajas:

- Alto rendimiento para gráficos.
- Control directo del renderizado.

Drag and Drop y SurfaceView en Android

¿Qué es Drag and Drop?

- Es una técnica de interacción que permite arrastrar (drag) y soltar (drop) elementos en la pantalla.

Implementación en Android:

- Uso de eventos táctiles (`onTouchEvent`).
- Manejo de coordenadas (X, Y) para actualizar la posición del objeto.
- Uso de `setOnDragListener()` en vistas estándar.

Relación con SurfaceView:

- En **SurfaceView** el control es manual:
  - Se capturan eventos táctiles.
  - Se redibuja el objeto en nuevas posiciones usando `Canvas`.

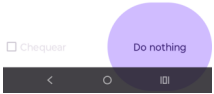
Ventajas:

- Ideal para juegos o interfaces dinámicas. Permite interacción intuitiva y mayor control en SurfaceView para animaciones en tiempo real.

Demo 17: Surface View

Del Repositorio Github:

- Copiar el archivo *MainActivity_Demo17.java* (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar los archivos *DragAndDropView.java*, *DragAndDropThread.java*, *Circulo.java*, *Figura.java y Rectangulo.java*, (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo *activity_main_demo17.xml* (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml
- Modificar dentro del archivo *activity_main_demo17.xml* la cadena *.com.example.a2026_01_holamundoandroid.DragAndDropView* por *<SuPackage>.DragAndDropView*.
- Modificar en el archivo *AndroidManifest.xml* la cadena *.MainActivity_Demo16* por *.MainActivity_Demo17*.



Clase: MainActivity_Demo18

- Es la actividad principal de la aplicación.
- Controla la interacción entre la interfaz y la lógica del juego.

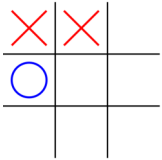
Componentes principales:

- **TicTacToeView:** Vista personalizada (probablemente basada en SurfaceView) para el juego.
- **Button (B1):** Botón para reiniciar el juego.

Funcionamiento:

- En `onCreate()`:
  - Se carga el layout con `setContentView()`.
  - Se inicializa el botón con `findViewById()`.
  - Se asigna un `OnClickListener`.
- Al presionar el botón:
  - Se invoca `resetGame()` en la vista.
  - Se reinicia el estado del juego (tablero, movimientos, etc.).

Demo 18: Tic-Tac-Toe



Del Repositorio Github:

- Copiar el archivo *MainActivity_Demo18.java* (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar los archivos *TicTacToeView.java*, (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo *activity_main_demo18.xml* (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml
- Modificar dentro del archivo *activity_main_demo18.xml* la cadena *.com.example.a2026_01_holamundoandroid.TicTacToeView* por *<SuPackage>.TicTacToeView*.
- Modificar en el archivo *AndroidManifest.xml* la cadena *.MainActivity_Demo17* por *.MainActivity_Demo18*.

Análisis de MainActivity_Demo19

Clase: MainActivity_Demo19

- Es la actividad principal de una aplicación Android.
- Extiende de `AppCompatActivity`.
- Se encarga de inicializar la interfaz y la vista del juego Pong (**PongSurfaceView:**).
- En `onCreate()`:
  - Se obtiene la referencia a `PongSurfaceView` con `findViewById()`.
  - **Ciclo de vida:**
    - `onResume()` y `onPause()` están definidos pero sin lógica adicional.
    - Normalmente se usarían para pausar/reanudar el hilo del juego.

Idea clave:

- La Activity actúa como contenedor, mientras que toda la lógica y renderizado del juego ocurre en `PongSurfaceView`.

Game Loop en PongSurfaceView (Android)

¿Qué es el Game Loop?

- Es un ciclo continuo que controla la ejecución de un videojuego.
- Se encarga de:
  - Actualizar la lógica del juego.
  - Dibujar los elementos en pantalla.
  - Controlar la velocidad (FPS).

Implementación en SurfaceView:

- Se ejecuta en un hilo independiente del UI thread.
- Usa **SurfaceHolder** para dibujar sobre el **Canvas**.

Métodos clave:

- **update():** Movimiento de pelota, colisiones, puntaje.
- **draw():** Dibujo en Canvas (paletas, pelota, fondo).
- **controlFPS():** Regula la velocidad del juego.

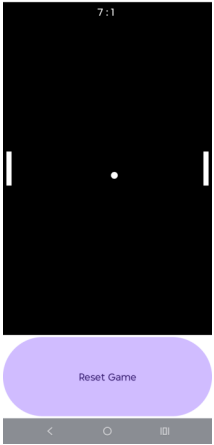
Idea clave:

- El Game Loop permite animaciones fluidas y control total del juego en tiempo real.

Demo 19: Pong Game

Del Repositorio Github:

- Copiar el archivo *MainActivity_Demo19.java* (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar los archivos *TicTacToeView.java*, (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo *activity_main_demo19.xml* (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml
- Modificar dentro del archivo *activity_main_demo19.xml* la cadena *.com.example.a2026_01_holamundoandroid.PongSurfaceView* por *<SuPackage>.PongSurfaceView*.
- Modificar en el archivo *AndroidManifest.xml* la cadena *.MainActivity_Demo18* por *.MainActivity_Demo19*.



Análisis de MainActivity_Demo20

Propósito general:

- Implementa un menú basado en botones para navegar entre múltiples actividades (Demo01–Demo19).
- Utiliza un solo **OnClickListener** para manejar todos los eventos.

Componentes principales:

- **Botones (B11–B54):** Representan una cuadrícula de opciones.
- **Tags:** Identificadores asociados a cada botón.

Funcionamiento:

- Se asigna un **tag** a cada botón (`"11"`, `"12"`, ..., `"53"`).
- Un **listener común** captura el clic:
  - Obtiene el tag del botón.
  - Usa un **switch** para decidir qué Activity abrir.
  - Lanza la nueva Activity con `Intent`.

Métodos auxiliares:

- **InicializarVistas():** Asocia variables con elementos XML.
- **InicializarTags():** Define identificadores para cada botón.

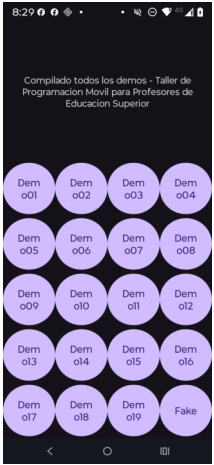
Demo 20: App Integradora

Del Repositorio Github:

- Copiar el archivo *MainActivity_Demo20.java* (dentro del Folder códigosJAVA) al mismo directorio donde esta el MainActivity.java
- Copiar el archivo *activity_main_demo20.xml* (dentro del Folder Carpeta_layout) al mismo directorio donde esta el activity_main.xml
- Modificar en el archivo *AndroidManifest.xml* la cadena *.MainActivity_Demo19* por *.MainActivity_Demo20*.

Agregar en el archivo AndroidManifest, despues de <\Activity> lo siguiente:

```
<!-- ES necesario registar todas las actividades adicionales-->
        <activity android:name=".MainActivity_Demo01"></activity>
        <activity android:name=".MainActivity_Demo02"></activity>
...
        <activity android:name=".MainActivity_Demo19"></activity>
```



Outline

1 Programación Móvil

2 Programación Orientada a Objetos

3 Configuracion de Dispositivo

4 Conceptos Basicos de Apps en Android

5 Demos

6 Conclusiones

Conclusiones

En este taller:

- Se explicaron los conceptos de Telefono Inteligente, Sistema Operativo.
- Se describieron los materiales necesarios (hardware y software) para la implementacion de una app en Android.
- Se revisaron los principales componentes de interfaz grafica y su uso en Apps de Android Studio.
- Se pusieron en practica aplicaciones simples que emplean componentes de interfaz grafica, manejo de archivos y librerias externas para propositos misceláneos.

¡Gracias!

Comentarios o Dudas: mnunom@upv.edu.mx